42. DNS Debugging and /etc/resolv.conf

42. DNS Debugging and /etc/resolv.conf

Phase: Phase 6 — Networking
Estimated time: 1 day
Slug: `42-networking-dns-deep`

Learning objectives

- Read resolv.conf and systemd-resolved
- Debug NXDOMAIN vs timeout
- Understand search domains
- Cache flush awareness

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

When an application says “cannot connect”, the first broken link is very often name resolution, not the network. A web service that works by IP but fails by name, a container that cannot reach a private API, a deploy that “randomly” hits an old server: all of these are DNS stories. If you can split a failure into “name did not resolve”, “name resolved to the wrong thing”, and “resolver did not answer”, you cut the search space by two thirds in under a minute.

Two answers matter most. **NXDOMAIN** means the authoritative side said the name does not exist (name-not-exist). A **timeout** means nobody answered at all, which is a path, firewall, or resolver-health problem. Treating them the same wastes hours.

Core concepts

Piece	What it does	Where to look
Stub resolver	libc code inside every program	`getent ahosts NAME`
nsswitch	order of sources: files, dns, mdns	`/etc/nsswitch.conf`
/etc/hosts	static overrides, checked first	`cat /etc/hosts`
resolv.conf	which nameservers and search domains	`/etc/resolv.conf`
systemd-resolved	local caching stub at 127.0.0.53	`resolvectl status`
Recursive resolver	does the walk for you, caches	`dig @1.1.1.1 NAME`
Authoritative server	owns the zone data	`dig +trace NAME`

Response codes you must know: NOERROR (ok, maybe empty answer), NXDOMAIN (name-not-exist), SERVFAIL (resolver or upstream broke), REFUSED (policy).

```
app --> libc --> nsswitch --> /etc/hosts ?
                     |
                     v
              resolv.conf / resolved (127.0.0.53)
                     |  cache hit? -> answer
                     v
              recursive resolver --> root --> TLD --> authoritative
                     ^                                    |
                     +----------- answer + TTL ----------+
```

Search domains: with `search corp.example` and `ndots:1`, a short name like `db` is tried as `db.corp.example` first. That is convenient and also the reason a typo can resolve to something surprising.

Worked examples

Example 1: compare what the program sees with what DNS says.

```
getent ahosts example.com        # what libc (your apps) resolves
dig +short example.com           # what DNS says via resolv.conf
```

If `getent` fails but `dig` works, suspect nsswitch, hosts, or a search domain, not the DNS server.

Example 2: NXDOMAIN versus timeout.

```
dig nosuchhost.example.com | grep status
# ;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN
dig @10.255.255.1 example.com +time=2 +tries=1
# ;; connection timed out; no servers could be reached
```

NXDOMAIN is a clean “no such name”: fix the name or the zone. The timeout says the server is unreachable: check route, firewall, UDP/TCP 53.

Example 3: who is the resolver?

```
resolvectl status | sed -n 1,20p
resolvectl query example.com
cat /etc/resolv.conf            # often only 127.0.0.53
```

Example 4: walk the delegation to find a stale record.

```
dig +trace www.example.com | tail -5
dig www.example.com +noall +answer      # note the TTL column
```

If the TTL says 3600, a changed record may stay stale up to an hour in caches. Lower the TTL before a planned cutover, not after.

Example 5: flush caches only after you have evidence.

```
resolvectl flush-caches
resolvectl statistics | grep -i cache
```

Mini-lab

1. Run `cat /etc/resolv.conf` and `resolvectl status`; write down the active nameserver and search domains.
2. Run `dig example.com` and `getent ahosts example.com`; confirm they agree.
3. Add a fake line to `/etc/hosts` in a lab VM: `203.0.113.9 lab.test`. Run `getent hosts lab.test` and `dig lab.test`. Explain why they differ.
4. Query a made-up name and record the status field. Say aloud what that status means (name-not-exist).
5. Point dig at an unroutable server with a short timeout and compare the failure to step 4.
6. Remove your hosts line. Note the TTL of one record and when it would expire.

Decision guide / failure modes

Symptom	Likely cause	First check
NXDOMAIN	typo, missing record, wrong search domain	`dig NAME +search` vs FQDN with dot
Timeout	resolver unreachable, port 53 blocked	`dig @server NAME +time=2`
SERVFAIL	DNSSEC failure, broken upstream	`dig +cd NAME`, try another resolver
Works by dig, fails in app	hosts file, nsswitch, container resolver	`getent ahosts NAME`
Old IP returned	cache or high TTL	`dig +trace`, `resolvectl flush-caches`
Private zone fails	hardcoded public resolver	remove 8.8.8.8 override

Failure modes to avoid: editing `/etc/resolv.conf` by hand on a host where systemd-resolved or NetworkManager rewrites it; declaring “network is down” because one lookup failed; skipping the trailing dot test for search-domain surprises. Rule of thumb: always ask the stub what the app sees, then ask the server what it knows, then walk the chain.

Common mistakes

- Hardcoding 8.8.8.8 breaking private zones
- Ignoring search domain effects
- Assuming dig failure equals network down

Exercises

Work in `exercises/42-networking-dns-deep/`. Read that folder’s `README.md` fully before coding.

```
cd exercises/42-networking-dns-deep
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement the `nxdomain_means` function so it prints one short token describing what an NXDOMAIN response tells you about the queried name. Use the vocabulary from the lesson: the token is lowercase, words joined with hyphens, no spaces, no trailing text. The function must not call the network; it is a recall check of the response-code table. After the test passes, run `dig` against a name you invent in your lab and read the `status:` field to confirm in practice. Then write one sentence in your notes that contrasts

NXDOMAIN with a timeout, and say which one points at the network path rather than the zone data.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/42-networking-dns-deep/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. NXDOMAIN meaning?
2. search domain effect?
3. Why dig +trace?

Further reading

- https://man7.org/linux/man-pages/man5/resolv.conf.5.html